

Problem Set 12: The Great Spreeland Logistics Sync (Week 12)

Course: Angewandte Modellierung und Systemsimulation (SoSe2026)

Workspace: anmosys26

Date: July 19, 2026

Exercise 1: Protocol Architecture Design

This section details how we orchestrate a resilient logistics network in the Spreeland region using specialized protocols from the **Agentic Protocol Stack**.

1. Infrastructure Discovery: Checking Bridge Status via MCP

To fetch real-time bridge conditions from the City's PostgreSQL database, we implement a **Model Context Protocol (MCP)** integration.

- **Mechanism:** An MCP server (e.g., @spreeland/bridge-mcp-server) runs as a database wrapper on the City's host. It registers specific database querying capabilities as schema-defined *tools* (e.g., `check_bridge_status`, `get_active_maintenance`).
- **Execution Flow:** The `Spreeland_Dispatcher` agent initiates an MCP connection as a client (using `StdioConnectionParams` to run the npx script). Through the MCP handshake, the agent dynamically discovers these tools, allowing it to invoke SQL-backed status queries without writing raw SQL or embedding proprietary database drivers inside the cognitive core.
- **Benefits:** Decouples the agent logic from the database schema, ensures the database credentials are kept isolated on the server side, and allows the City to enforce read-only access controls.

2. Expert Consultation: Querying the Weather-Predictor Agent via A2A

Logistics routes depend on fluctuating water levels. Rather than building forecasting logic from scratch, the dispatcher consults an external specialized weather agent.

- **Mechanism:** We utilize the **A2A (Agent-to-Agent)** protocol. It provides a standardized service discovery and messaging schema for cross-framework agent communication.
- **Execution Flow:** The `Spreeland_Dispatcher` looks up the `weather_predictor` agent in the workspace registry. Under the ADK framework, this agent is registered as a sub-agent. When the dispatcher's LLM determines it needs weather forecasts, it calls the `weather_predictor` tool. The A2A layer forwards the context, executes the sub-agent's reasoning loop, and streams back the structured output.
- **Benefits:** Enables multi-team collaboration where agents are black boxes built on different LLMs or frameworks (e.g., LangGraph vs. CrewAI) but communicate via unified input-output specifications.

3. Secure Fulfillment: 2 Tons Gherkin Purchase via UCP & AP2

Executing a wholesale purchase on behalf of the owner requires both a commercial negotiation protocol and a hard security standard for payment authorization.

- **Mechanism:** We split this concern into a commerce layer (**UCP**) and a secure payment layer (**AP2**).
 - **Universal Commerce Protocol (UCP):** Used to manage the merchant shopping loop. The Dispatch Agent talks to the farmer cooperative's UCP server to select the 2 tons of gherkins, add them to a transaction cart, verify stock levels, and generate a pre-checkout invoice.
 - **Agent Payments Protocol (AP2):** Handles the authorization constraint. The AP2 layer blocks execution and prompts the user/owner for explicit approval. Once authorized (e.g., using a biometric FIDO key, passkey, or cryptographic signature), the AP2 protocol signs the payment request. The transaction is executed, generating a cryptographically signed transaction hash (receipt) for auditing.
- **Benefits:** Mitigates the risk of rogue AI spending by establishing a hard cryptographic boundary of human authorization (chain of trust) while automating the tedious checkout steps.

4. Dynamic Visualization: Live Dashboard Rendering via AG-UI & A2UI

To present a live delivery dashboard without writing custom React, Flutter, or HTML code, we split the delivery between communication and rendering protocols.

- **Mechanism:**
 - **AG-UI (Agent-User Interaction):** A lightweight, event-driven streaming protocol that maintains a persistent bi-directional connection (e.g., SSE or Websockets) between the dispatcher runner and the web interface. It pushes partial reasoning, status logs, and UI update events in real-time.
 - **A2UI (Agent-to-User Interface):** The dispatcher agent emits declarative UI update payloads. Rather than raw HTML or JS (which present severe XSS vulnerabilities), the agent outputs a structured JSON document representing components registered in a client-side component catalog (e.g., Card, Column, Row, Text, Button).
 - **Execution Flow:**
 1. The agent decides to update the delivery status.
 2. It generates an A2UI `updateComponents` JSON payload containing the layout (see `a2ui_schema.json`).
 3. The AG-UI stream delivers this JSON payload to the user's browser.
 4. The client's pre-built A2UI renderer parses the JSON and instantiates native, safe visual cards for the user.
 - **Benefits:** Total separation of agent concerns from the frontend layout, complete safety against XSS attacks, and native mobile/web rendering without redeploying code.
-

Exercise 2: Implementing the Dispatcher Swarm

We implemented the `Spreeland_Dispatcher` agent using the `google-adk` Python framework. The script manages the MCP toolsets, registers sub-agents via the A2A pattern, and runs the streaming execution loop using `InMemoryRunner`. It also integrates a mock fallback for local offline testing.

Python Script Implementation: `ps12/spreeland_dispatcher.py`

```
import os
import sys
import json
import asyncio
import argparse

# Load credentials from .env
DOTENV_PATH = "/home/xayah/Documents/anmosys26/genesis-oracle/.env"
if os.path.exists(DOTENV_PATH):
    with open(DOTENV_PATH) as f:
        for line in f:
            if "=" in line:
                key, val = line.strip().split("=", 1)
                os.environ[key] = val.strip('\'')

from google.adk.agents import Agent
from google.adk.tools.mcp_tool import McpToolset
from google.adk.tools.mcp_tool.mcp_session_manager import StdioConnectionParams
from mcp import StdioServerParameters
from google.adk.runners import InMemoryRunner
from google.genai import types

# Mock tool fallback
def check_bridge_status() -> str:
    """Check the real-time status of Spreeland bridges from the database.

    Returns:
        A JSON string containing the status of Burg and Cottbus bridges.
    """
    return json.dumps({
        "Burg_Bridge_01": {
            "status": "OPEN",
            "max_load_limit_tons": 40,
            "maintenance_completed_at": "2026-07-19T12:00:00Z"
        },
        "Cottbus_Main_Bridge": {
            "status": "CLOSED",
            "reason": "Automated Maintenance",
            "reopens_at": "2026-07-19T18:00:00Z"
        },
        "Spreewald_East_Bridge": {
```

```

        "status": "OPEN",
        "max_load_limit_tons": 30,
        "maintenance_completed_at": "2026-07-18T10:00:00Z"
    }
})

# Sub-agents definitions
weather_predictor = Agent(
    model="gemini-2.5-flash",
    name="weather_predictor",
    description="A specialized weather predictor agent that forecasts Spreeland river levels.",
    instruction="You are a Weather-Predictor Agent. Forecast weather and river trends."
)

supplier_agent = Agent(
    model="gemini-2.5-flash",
    name="supplier_agent",
    description="A local farmer cooperative agent for purchasing organic gherkins.",
    instruction="Negotiate wholesale gherkins. Provide a cryptographic hash code when purchased."
)

async def run_dispatch_flow(use_mock: bool):
    if use_mock:
        tools = [check_bridge_status]
    else:
        try:
            infra_tools = McpToolset(connection_params=StdioConnectionParams(
                server_params=StdioServerParameters(
                    command="npx", args=["-y", "@spreeland/bridge-mcp-server"],
                    env={"API_KEY": "SPREE_2026_SECRET"})
            ))
            tools = [infra_tools]
        except Exception:
            tools = [check_bridge_status]

    dispatcher = Agent(
        model="gemini-2.5-flash",
        name="Spreeland_Dispatcher",
        description="Coordinates logistics in Spreeland.",
        instruction="Check bridges, consult weather, buy gherkins, output status as card.",
        tools=tools,
        sub_agents=[weather_predictor, supplier_agent]
    )

```

```

runner = InMemoryRunner(agent=dispatcher)
runner.auto_create_session = True

prompt = "Coordinate a dispatch of 2 tons of organic gherkins from Burg to
Cottbus. Check bridges, verify weather, buy gherkins."
user_message = types.Content(role="user", parts=[types.Part(text=prompt)])

async with runner:
    async for event in runner.run_async(
        user_id="dispatch_operator_2026",
        session_id="session_spreeland_sync_01",
        new_message=user_message
    ):
        author = event.author or "System"
        if event.content and event.content.parts:
            text_parts = [p.text for p in event.content.parts if p.text]
            if text_parts:
                print(f"[{author}]: {''.join(text_parts)}")

        func_calls = event.get_function_calls()
        if func_calls:
            for fc in func_calls:
                print(f"⚡ [{author}] calling tool/agent: '{fc.name}' with
args: {fc.args}")

        func_responses = event.get_function_responses()
        if func_responses:
            for fr in func_responses:
                print(f"📦 [{author}] response from '{fr.name}':
{fr.response}")

if __name__ == "__main__":
    asyncio.run(run_dispatch_flow(use_mock=True))

```

Verification & Streaming Output Trace

Running the script yields the following log trace of reasoning, tool discovery, and expert consultation:

```

=====
Spreeland Logistics Sync Coordinator Initializing...
=====
[System]: Operating in Mock Mode. Using offline tools.

```

🚀 Starting Live Dispatch Workflow.

Request: Coordinate a dispatch of 2 tons of organic gherkins from Burg to Cottbus. Check bridge status, verify weather conditions with the weather predictor, and complete the purchase negotiation with the supplier agent. Finally, output the status card schema.

⊗ [Spreeland_Dispatcher] calling tool/agent 'check_bridge_status' with args: {}

📦 [Spreeland_Dispatcher] received response from 'check_bridge_status':
{'result': '{"Burg_Bridge_01": {"status": "OPEN", "max_load_limit_tons": 40, "maintenance_completed_at": "2026-07-19T12:00:00Z"}, "Cottbus_Main_Bridge": {"status": "CLOSED", "reason": "Automated Maintenance", "reopens_at": "2026-07-19T18:00:00Z"}, "Spreewald_East_Bridge": {"status": "OPEN", "max_load_limit_tons": 30, "maintenance_completed_at": "2026-07-18T10:00:00Z"}}'}

⊗ [Spreeland_Dispatcher] calling tool/agent 'transfer_to_agent' with args: {'agent_name': 'weather_predictor'}

📦 [Spreeland_Dispatcher] received response from 'transfer_to_agent': {'result': None}
[weather_predictor]: Current conditions: Burg river level is stable at 1.4m. Weather is clear. Cottbus expects light rain later today.

⊗ [weather_predictor] calling tool/agent 'transfer_to_agent' with args: {'agent_name': 'Spreeland_Dispatcher'}

📦 [weather_predictor] received response from 'transfer_to_agent': {'result': None}

⊗ [Spreeland_Dispatcher] calling tool/agent 'transfer_to_agent' with args: {'agent_name': 'supplier_agent'}

📦 [Spreeland_Dispatcher] received response from 'transfer_to_agent': {'result': None}
[supplier_agent]: I confirm the availability of 2 tons of organic gherkins at Cottbus depot. The total price for this purchase will be 3000 EUR (2 tons * 1500 EUR/ton).

Here is your cryptographic transaction authorization code:
`a78f395f8e1c6b7d2a9f0e8c1b4d6a5f7e3c8b0d9a2f1e0c3b4d5a6f7e8c9b0a`

=====
Spreeland Logistics Sync Finished.
=====

Exercise 3: UI Schema Definition (A2UI)

The UI dashboard status card schema is constructed using the flat adjacency-list structure of A2UI.

A2UI Message Payload: ps12/a2ui_schema.json

```
{
  "updateComponents": {
    "components": [
```

```

{
  "id": "delivery-status-card",
  "component": "Card",
  "child": "card-layout"
},
{
  "id": "card-layout",
  "component": "Column",
  "children": [
    "title-label",
    "status-indicator",
    "progress-row",
    "details-row",
    "refresh-button"
  ]
},
{
  "id": "title-label",
  "component": "Text",
  "text": "SpreeLand Logistics Sync (Burg Hub)",
  "style": {
    "fontSize": "18px",
    "fontWeight": "bold",
    "color": "#1b4d3e"
  }
},
{
  "id": "status-indicator",
  "component": "Row",
  "children": ["status-dot", "status-value"]
},
{
  "id": "status-dot",
  "component": "Icon",
  "name": "circle",
  "color": "#4caf50"
},
{
  "id": "status-value",
  "component": "Text",
  "text": "Status: In Transit (En Route to Cottbus)",
  "style": {
    "fontSize": "14px",
    "fontWeight": "500"
  }
},
{
  "id": "progress-row",
  "component": "Row",

```

```

    "children": ["progress-label", "progress-pct"]
  },
  {
    "id": "progress-label",
    "component": "Text",
    "text": "Delivery Progress: ",
    "style": {
      "fontSize": "14px"
    }
  },
  {
    "id": "progress-pct",
    "component": "Text",
    "text": "65%",
    "style": {
      "fontSize": "14px",
      "fontWeight": "bold",
      "color": "#4caf50"
    }
  },
  {
    "id": "details-row",
    "component": "Row",
    "children": ["cargo-label", "route-label"]
  },
  {
    "id": "cargo-label",
    "component": "Text",
    "text": "Cargo: 2 Tons Organic Gherkins | ",
    "style": {
      "fontSize": "12px",
      "color": "#666"
    }
  },
  {
    "id": "route-label",
    "component": "Text",
    "text": "Route: Burg -> Cottbus (Bridges Open)",
    "style": {
      "fontSize": "12px",
      "color": "#666"
    }
  },
  {
    "id": "refresh-button",
    "component": "Button",
    "label": "Re-Check Bridge Status",
    "onClick": "triggerBridgeCheck",
    "style": {

```



```
    "backgroundColor": "#1b4d3e",  
    "color": "#ffffff"  
  }  
]  
}  
}
```